SAiDL Summer Assignment 2026

Your Name Here

May 2026

Contents

1 Core ML: Long-Context Sequence Modeling

1.1 Overview

In this section, we design and evaluate a modular long-context sequence model on the WikiText-2 dataset. We systematically compare attention variants, positional encodings, and convolutional hybrid architectures.

1.2 Baseline Transformer

We implement a standard Transformer following Vaswani et al. (2017) with the following configuration:

- Embedding dimension: 256
- Number of heads: 8
- Number of layers: 4
- Feedforward dimension: 1024
- Context length: 512
- Parameters: 29,022,208

1.3 Attention Variants

We implement and evaluate three attention variants as replacements for standard multi-head attention.

1.3.1 Multi-Query Attention (MQA)

Instead of each head maintaining independent key and value projections, MQA shares a single K and V across all heads. This reduces parameters and memory usage while maintaining competitive performance.

1.3.2 Sliding Window Attention (SWA)

Each token attends only to the W=64 nearest preceding tokens. This limits long-range dependencies but reduces computational cost for local pattern recognition.

1.3.3 Linear Attention

Replaces the softmax attention with a kernel-based approximation using ELU+1 as the feature map. By reordering matrix multiplications as $\phi(Q)(\phi(K)^T V)$, the T×T attention matrix is avoided entirely, improving throughput.

Model	Perplexity	Parameters	Throughput (tok/s)
Baseline	578.83	29,022,208	22,571
MQA	426.02	28,561,664	22,905
SWA	587.46	29,022,208	22,958
Linear	559.04	29,022,208	26,539
ALiBi	476.94	29,022,208	22,109
RoPE	481.29	29,022,208	22,066
Conv+RoPE	481.18	29,022,208	22,053

Table 1: Comparison of attention variants and positional encodings on WikiText-2

1.4 Results

1.5 Analysis

MQA achieves the best perplexity (426.02) by sharing K and V across heads, effectively concentrating learning capacity. Linear Attention achieves the highest throughput (26,539 tok/s) by avoiding the quadratic T×T attention matrix. SWA performs slightly worse than baseline because restricting the window to 64 tokens prevents the model from capturing long-range dependencies present in WikiText-2. ALiBi and RoPE both significantly outperform the baseline, with ALiBi's distance penalty providing strong inductive bias for language modeling. The Conv+RoPE hybrid shows minimal improvement over RoPE alone, suggesting that for this dataset and context length, the convolutional local features do not add significant information beyond what attention already captures.

1.6 Positional Encodings

We evaluate three positional encoding schemes:

- **ALiBi**: Adds a linear distance penalty to attention scores. No learned parameters, generalizes to unseen lengths.
- **RoPE**: Rotates Q and K vectors by position-dependent angles. Encodes relative positions implicitly, used in Llama and Mistral.
- **Baseline**: Learned positional embeddings. Cannot extrapolate beyond training length.

1.7 Convolutional Hybrid

We add a depthwise separable Conv1D layer before each attention block to capture local n-gram patterns. Combined with RoPE, this architecture achieves perplexity of 481.18, nearly identical to RoPE alone (481.29), suggesting attention already captures local patterns sufficiently at this scale.

2 Sparsity & Optimization

2.1 Overview

We compare LoRA, AdaLoRA, and a custom SoRA implementation on the CoLA task from the GLUE benchmark using BERT-base-uncased as the backbone.

2.2 Methods

2.2.1 LoRA

LoRA freezes pretrained weights W_0 and learns a low-rank update $W = W_0 + BA$ where $A \in R^{r \times d}$ and $B \in R^{k \times r}$. We apply LoRA to the query, key, and value projections with rank $r = 8$.

2.2.2 AdaLoRA

AdaLoRA dynamically allocates rank during training based on importance scores. Unlike LoRA's fixed rank, AdaLoRA can concentrate capacity in important layers.

2.2.3 SoRA

SoRA introduces a gating vector g with ℓ_1 regularization:

$$\mathcal{L}(\Delta) = \mathcal{L}_0(\Delta) + \lambda \sum_k ||g^{(k)}||_1$$

The proximal update applies soft-thresholding to drive small gate values to exactly zero, inducing true sparsity.

2.3 Results

Method	Best MCC	Trainable Params	% of Total	Time (s)
LoRA	0.513	443,906	0.40%	314.9
AdaLoRA	0.410	665,522	0.60%	319.1
SoRA	0.236	296,642	0.27%	186.3

Table 2: Comparison of PEFT methods on CoLA (BERT-base-uncased)

2.4 Analysis

LoRA achieves the best MCC (0.513) with a moderate parameter count. AdaLoRA uses more parameters than LoRA despite adaptive rank allocation, suggesting the dynamic ranking did not find a more efficient solution within 3 epochs.

SoRA uses the fewest parameters and trains fastest, but achieves lower MCC — likely because the proximal updates require more epochs to converge as gates are gradually driven to zero.

2.5 SGD vs Proximal Gradient (Q2)

2.5.1 SGD Update Rule

The explicit SGD update for gating vector $g^{(k)}$ with ℓ_1 subgradient is:

$$g^{(k)} \leftarrow g^{(k)} - \eta\lambda \cdot \text{sign}(g^{(k)})$$

2.5.2 Proximal Soft-Thresholding Update

The proximal update applies soft-thresholding:

$$g^{(k)} \leftarrow \text{sign}(g^{(k)}) \cdot \max(|g^{(k)}| - \lambda\eta, 0)$$

2.5.3 Key Differences

The proximal update sets values below the threshold to *exactly* zero, producing true sparsity. SGD with ℓ_1 subgradients only shrinks values toward zero but never reaches it exactly. Our numpy and PyTorch implementations confirm this: given $g = [0.5, -0.3, 0.01, -0.001, 0.8]$, proximal produces one exact zero while SGD produces none.

Regarding subgradient choice: at $g = 0$, $\partial|g|$ is not uniquely defined — any value in $[-1, 1]$ is valid. The proximal operator avoids this ambiguity entirely by operating on the closed-form solution. This makes proximal theoretically cleaner and practically more stable for inducing sparsity.

2.6 Low-Rank Adaptation for xLSTM and Mamba (Q3)

2.6.1 xLSTM

For xLSTM, LoRA is applied to the gate weight matrices (input gate W_i, forget gate W_f) since these control information flow through the recurrent cell. The adaptation is:

$$W_i = W_i^0 + B_i A_i, \quad W_f = W_f^0 + B_f A_f$$

Our implementation achieves 1,088 trainable parameters out of 17,472 total (6.2%).

2.6.2 Mamba

For Mamba SSMs, LoRA is applied to the B and C projection matrices which control input-output mappings of the state space. The state transition matrix A is left unchanged as it encodes the core dynamics.

$$B = B^0 + B_b B_a, \quad C = C^0 + C_b C_a$$

Our implementation achieves 640 trainable parameters out of 2,768 total (23.1%).